

Practical Lesson 1

Plotly fundamentals

MECD 2024/25

Visualização Avançada de Dados

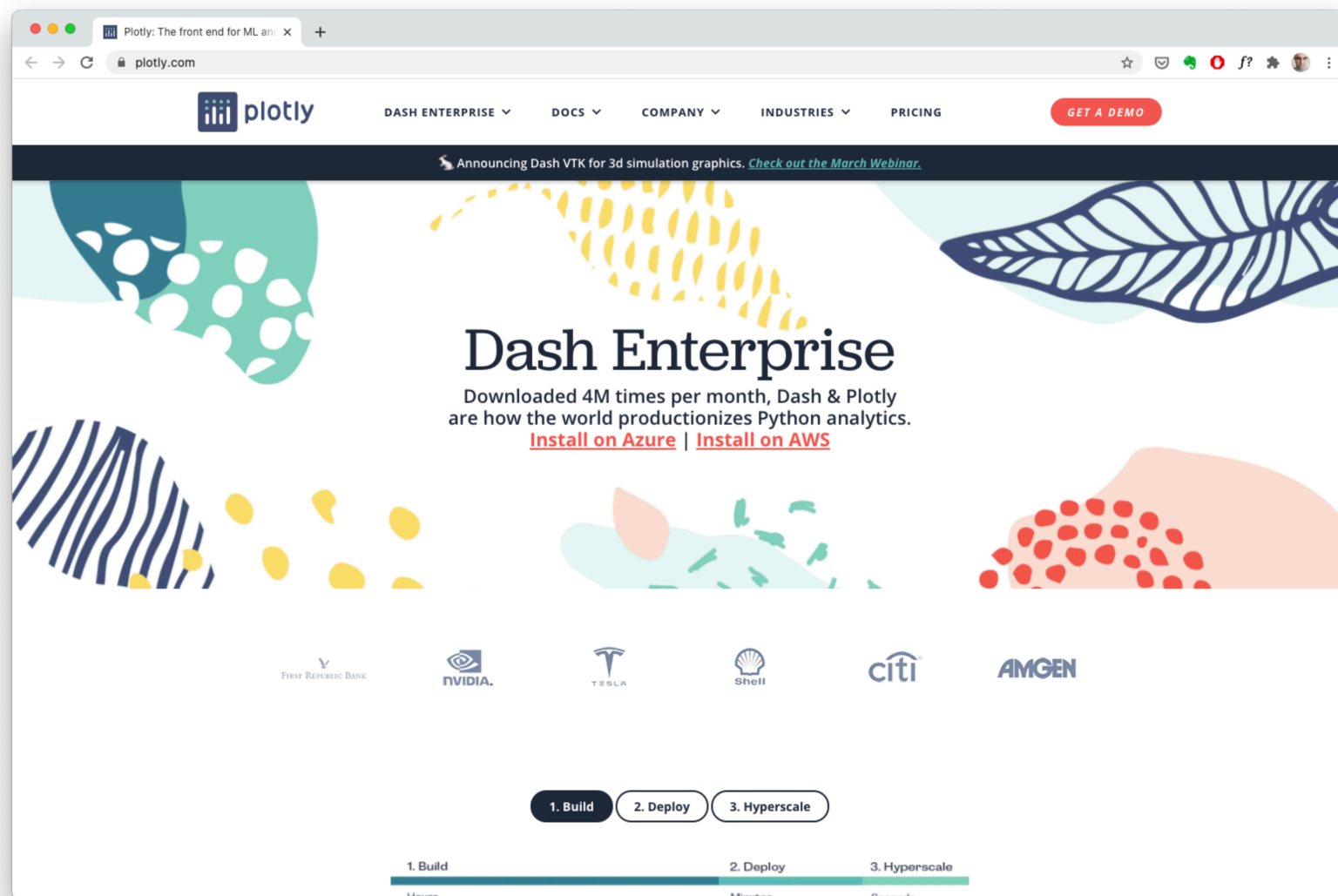
Evgheni Polisciuc

Outline

- 1. Introduction**
2. The figure structure
3. Graph objects vs Express
4. Creating and updating figures
5. Displaying figures
6. Plot from CSV
7. Scatter plots
8. Line charts
9. Multiple series

Plotly overview

"The plotly Python library is an interactive, open-source plotting library that supports over 40 unique chart types covering a wide range of statistical, financial, geographic, scientific, and 3-dimensional use-cases."



Plotly overview

Domain situation

- **Python** library (also exist for R and JavaScript);
- Open-source;
- Built on top of the Plotly JavaScript which uses D3.js to render graphics;
- Used to create, manipulate or render graphical figures;
- Ability to be displayed in Jupiter, saved as HTML pages, or **served as part of pure Python-built web applications using Dash**;
- Integrates a multitude of visualization models.

Outline

1. Introduction
- 2. The figure structure**
3. Graph objects vs Express
4. Creating and updating figures
5. Displaying figures
6. Plot from CSV
7. Scatter plots
8. Line charts
9. Multiple series

Figure Data Structure

- The visualization models are represented by **Data Structures** (referred to as Figures)
- Figures can be represented as
 - Python dictionaries (`dict`)
 - Instances of `plotly.graph_objects.Figure` class
- To view an underlying data structure use `print(fig)`
- Figures support `fig.to_dict()` and `fig.to_json()` methods

Tree of attributes

Figure Data Structure

- Figures represented as trees with nodes called **"attributes"**
- The root node contains three top-level attributes:
 - **data**
 - **layout**
 - frames
- Attributes can be set in two ways:
 - Pythonic style: `fig.layout.title.font.family="Open Sans"`
 - Using update method: `fig.update_layout(title_font_family="Open Sans")`
note the use of underscores.
- It is not necessary to populate every attribute of every object

Full reference of the attributes can be found here: <https://plotly.com/python/reference/index/>

Data attribute (traces)

Figure Data Structure

- Top-level attribute
- Its value must be a list of dicts, which are called “**traces**”
- **Trace**
 - Represents a set of related **graphical marks**
 - Has multiple possible **types** (40+) (e.g., bar, scatter, pie, etc.)
 - Must have a **type**, which defines other allowable attributes
 - Drawn on a single subplot
 - Must have a single legend entry
- Example

```
import plotly.graph_objects as go

trace0 = go.Scatter(...)

trace1 = go.Bar(...)

fig = go.Figure(data=[trace0, trace1])
```


Layout **attribute**

Figure Data Structure

- Top-level attribute
- Must be a `dict`
- Control positioning
- Configure non-data-related parts of visualization
 - Dimensions and margins (bounds)
 - Figure-wide defaults: templates, fonts, colors, hover-label, and modebar defaults
 - Title and legend
 - Color axis and color bars
 - Subplots
 - Non-data marks, such as annotations, shapes, and images
 - Controls (e.g., `layout.sliders`)

Frame attribute

Figure Data Structure

- Top-level attribute
- Must be a list of dicts, which define sequential
- Each frame contains its own data attribute
- Usually controlled via controls defined in `layout.slider` or `layout.updatemenus`

config attribute

Figure Data Structure

- Controls certain figure behaviors in render-time
- Exposed as the config argument of the `.show()` method
- Example:

```
config = dict({'scrollZoom': True})  
  
fig.show(config=config)
```

Config has its own documentation: <https://plotly.com/python/configuration-options/>

Outline

1. Introduction
2. The figure structure
- 3. Graph objects vs Express**
4. Creating and updating figures
5. Displaying figures
6. Plot from CSV
7. Scatter plots
8. Line charts
9. Multiple series

Graph objects

- Python module: `plotly.graph_objects` (usually imported as `go`)
- Automatically-generated hierarchy of Python classes
- The “graph objects” refer to instances of these classes
- The primary classes are `Figure` and `FigureWidget`, which represent entire visualizations
- Contains many methods for manipulating their attributes or rendering (e.g., `.update_layout()`, `.add_trace()`, `.show()`, `.to_json()`)

Graph objects

And magic underscore notation

- Non-leaf attributes are represented by instances of a class in `plotly.graph_objects`
- Example:
 - `fig.layout` is an object of `plotly.graph_objects.Layout`
 - `fig.layout.margin` is an object of `plotly.graph_objects.layout.Margin`
- Use of “magic underscore” notation
 - Ex: `go.Figure(layout_margin=dict(t=10, b=10, r=10, l=10))`

Graph objects or Dictionaries?

- Graph objects provide precise data validation
- Graph objects contain descriptions of each valid property as Python docstrings
- Properties of graph objects can be accessed using both Pythonical styles
 - Key lookup (e.g. `fig["layout"]`)
 - Property access (e.g. `fig.layout`).
- Graph objects support functions for updating already constructed figures (`.update_layout()`, `.add_trace()` etc).
- Graph object constructors and update methods accept "magic underscores". Ex.
 - `go.Figure(layout_title_text="The Title")`
 - vs.
 - `dict(layout=dict(title=dict(text="The Title")))`.

Express

Express

- Python module: `plotly.express` (usually imported as `px`)
- Ability to create entire figures at once
- High-level API comparing to Graph objects
- Express functions use Graph objects internally and return `plotly.graph_objects.Figure`
- Any figure can be created with graph objects

Express

High-level features

- Single entry point into plotly
- Returns `plotly.graph_objects.Figure`
- Flexible input formats (e.g., lists, dicts, pandas DataFrame, GeoPandas, numpy)
- Automatic trace and layout configuration
- Automatic figure and hover labeling
- Styling control and uniform color handling
- Animations
- ...

Outline

1. Introduction
2. The figure structure
3. Graph objects vs Express
- 4. Creating and updating figures**
5. Displaying figures
6. Plot from CSV
7. Scatter plots
8. Line charts
9. Multiple series

Create

Using express

```
import plotly.express as px
```

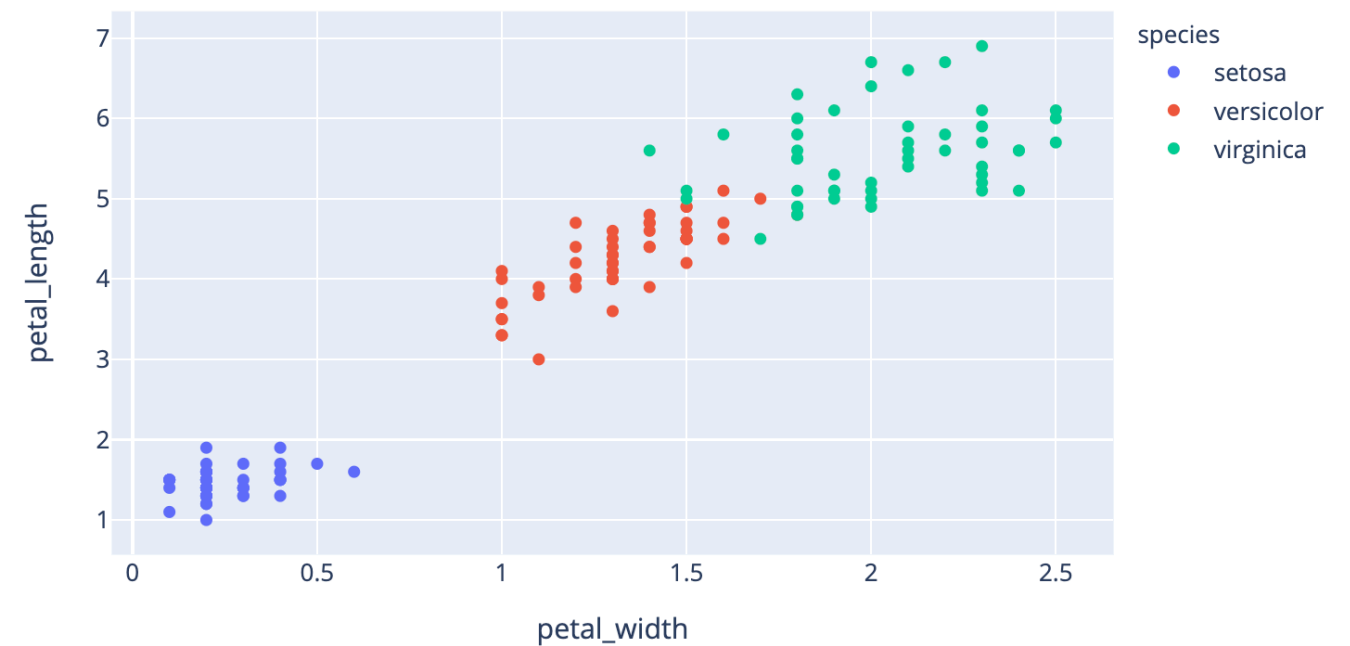
```
df = px.data.iris()
```

```
fig = px.scatter(df,  
                x="petal_width",  
                y="petal_length",  
                color="species",  
                title="Creating figures using Express")
```

```
# Print fig to reveal internal data structure  
# print(fig)
```

```
fig.show()
```

Creating figures using Express



Create

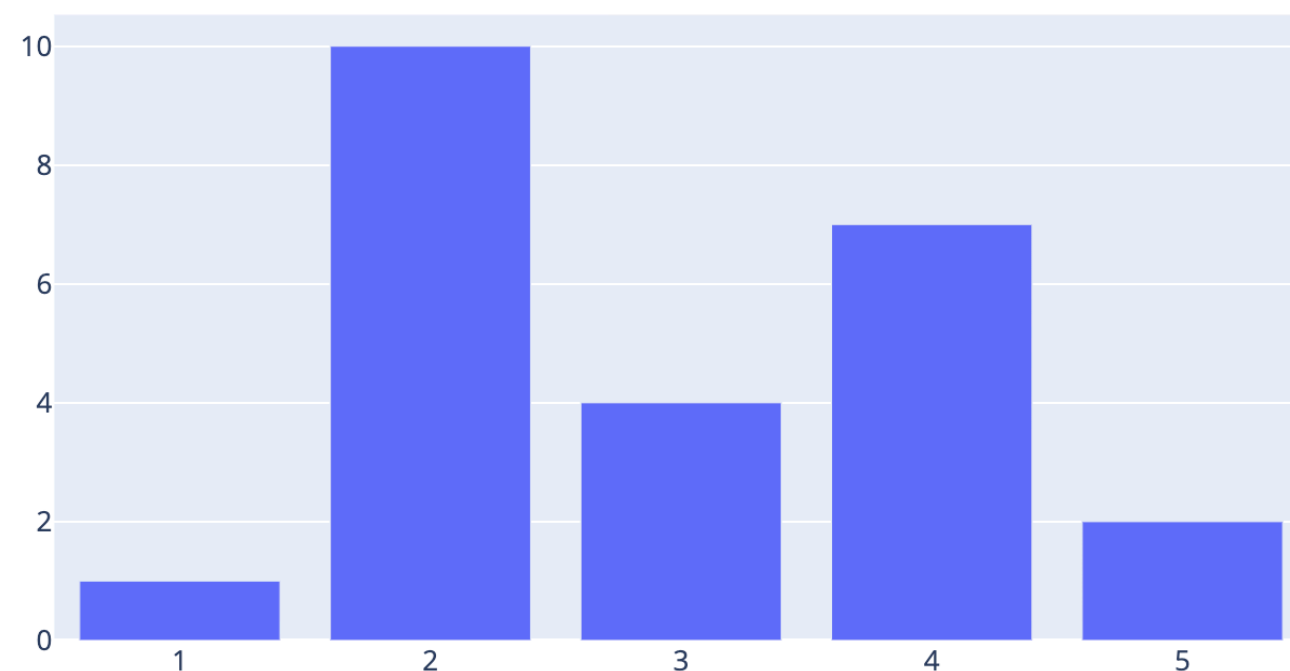
Using graph objects

```
import plotly.graph_objects as go
```

```
fig = go.Figure(  
    data=[go.Bar(x=[1, 2, 3, 4, 5], y=[1, 10, 4, 7, 2])],  
    layout=dict(title=dict(text="Creating figures using Graph Objects"))  
)
```

```
fig.show()
```

Creating figures using Graph Objects



Create

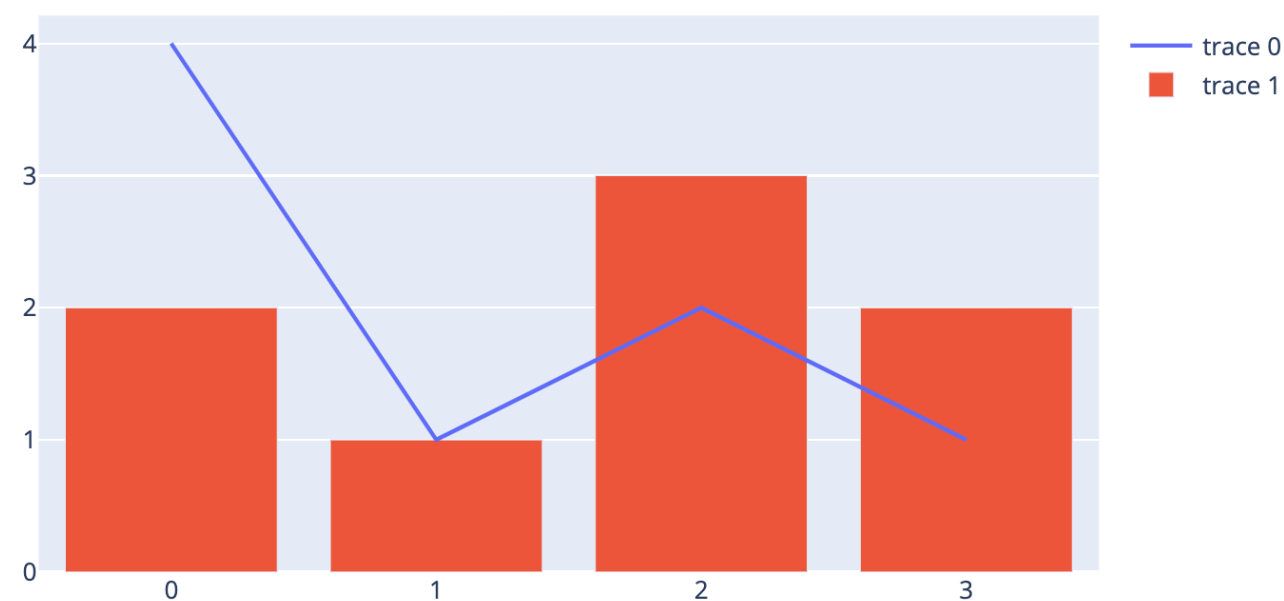
Add trace

```
import plotly.graph_objects as go

fig = go.Figure()
fig.add_trace(go.Scatter(y=[4, 1, 2, 1], mode="lines"))
fig.add_trace(go.Bar(y=[2, 1, 3, 2]))

fig.show()
```

Warning! **Visualization crime!**



Create

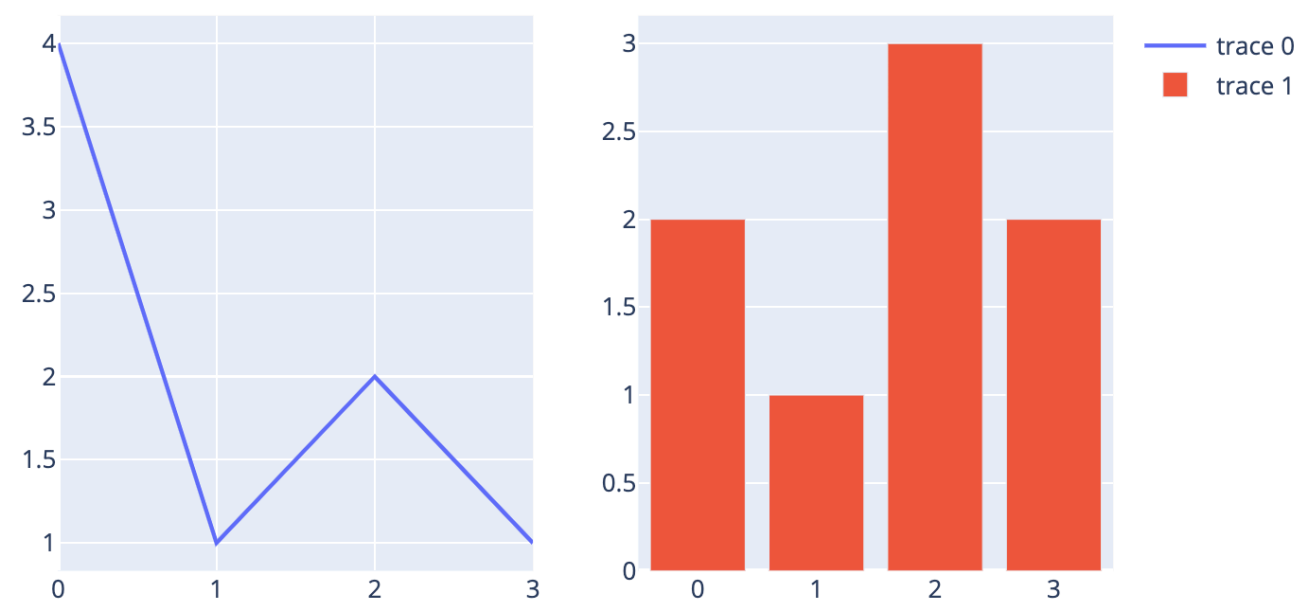
Subplots

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go
```

```
fig = make_subplots(rows=1, cols=2)
```

```
fig.add_trace(go.Scatter(y=[4, 1, 2, 1], mode="lines"), row=1, col=1)
fig.add_trace(go.Bar(y=[2, 1, 3, 2]), row=1, col=2)
```

```
fig.show()
```



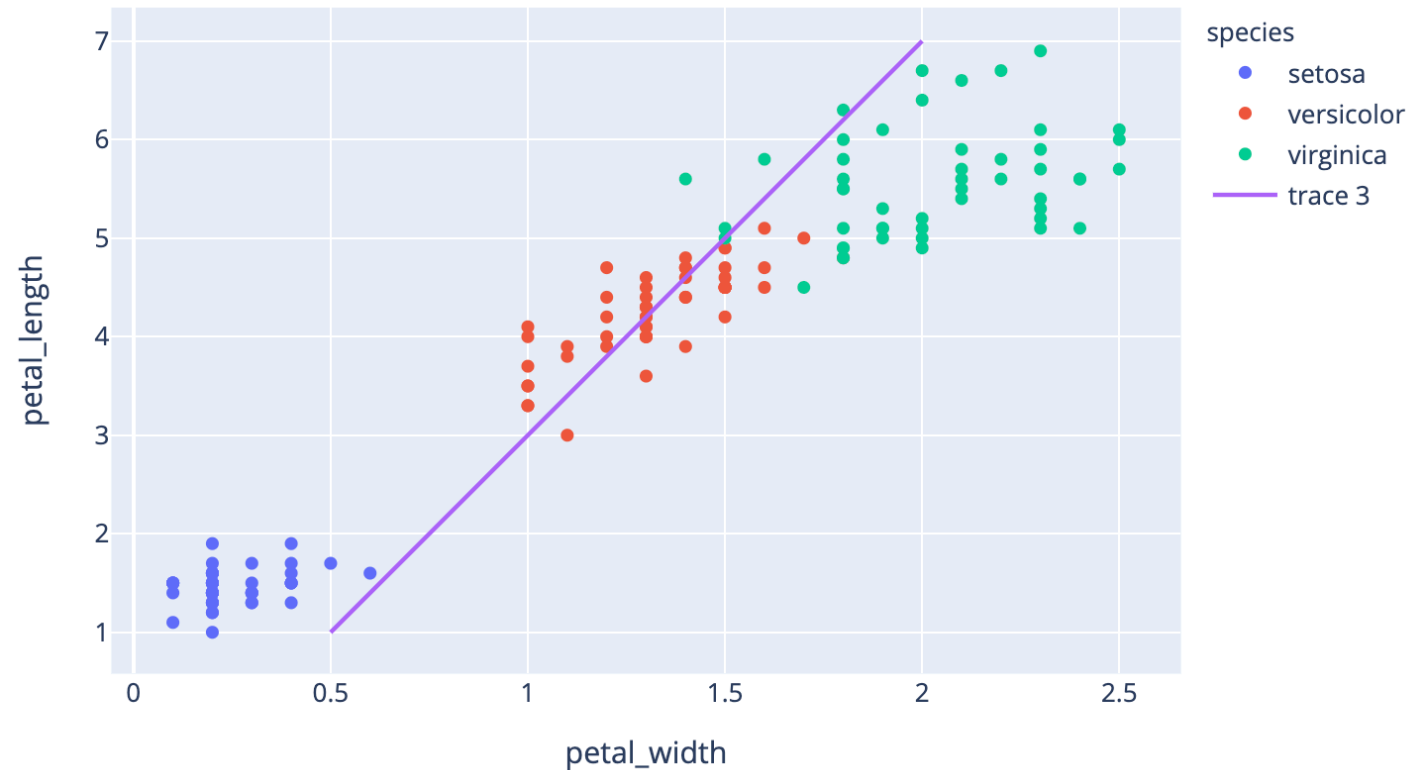
Create

Subplots

```
import plotly.express as px
```

```
df = px.data.iris()
fig = px.scatter(df,
                 x="petal_width",
                 y="petal_length",
                 color="species")
```

```
fig.add_trace(go.Scatter(x=[0.5, 2], y=[1, 7], mode="lines"))
fig.show()
```



Outline

1. Introduction
2. The figure structure
3. Graph objects vs Express
4. Creating and updating figures
- 5. Displaying figures**
6. Plot from CSV
7. Scatter plots
8. Line charts
9. Multiple series

Create

.show()

```
...  
fig.show()  
  
...  
fig.show(renderer="png")  
  
# for static image renderers need orca module installed
```

Outline

1. Introduction
2. The figure structure
3. Graph objects vs Express
4. Creating and updating figures
5. Displaying figures
- 6. Plot from CSV**
7. Scatter plots
8. Line charts
9. Multiple series

1. Import data

Pandas

```
import pandas as pd
```

```
df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/2014_apple_stock.csv')
```

```
df.head()
```

Plotly has a lot of build-in datasets! <https://plotly.com/python-api-reference/generated/plotly.data.html#module-plotly.data>

2. Plot

Using Express

```
import pandas as pd
import plotly.express as px

df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/2014_apple_stock.csv')

fig = px.line(df,
              x = 'AAPL_x',
              y = 'AAPL_y',
              title='Apple Share Prices over time (2014)')

fig.show()
```

2. Plot

Using Express

```
import pandas as pd
import plotly.graph_objects as go
```

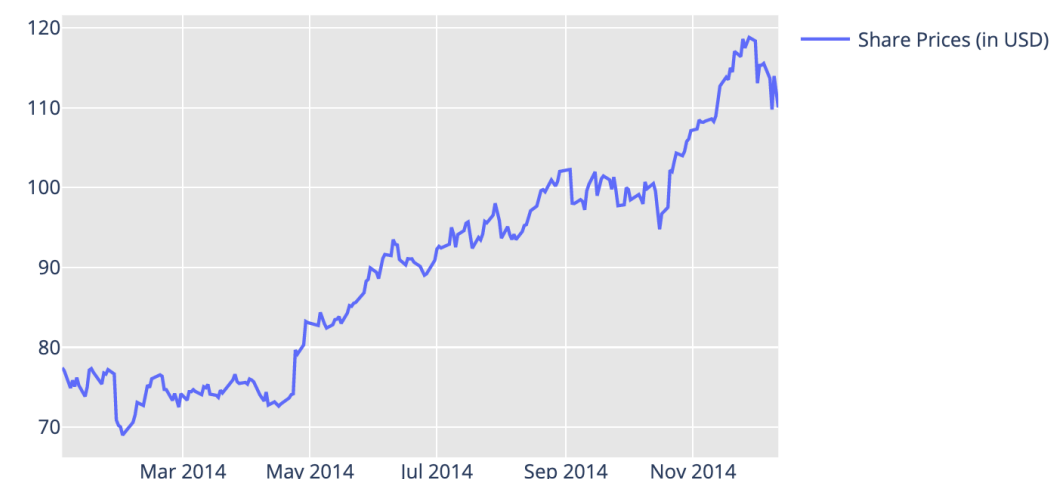
```
df = pd.read_csv('https://raw.githubusercontent.com/plotly/datasets/master/2014_apple_stock.csv')
```

```
fig = go.Figure(go.Scatter(x = df['AAPL_x'], y = df['AAPL_y'],
                           name='Share Prices (in USD)'))
```

```
fig.update_layout(title='Apple Share Prices over time (2014)',
                  plot_bgcolor='rgb(230, 230,230)',
                  showlegend=True)
```

```
fig.show()
```

Apple Share Prices over time (2014)



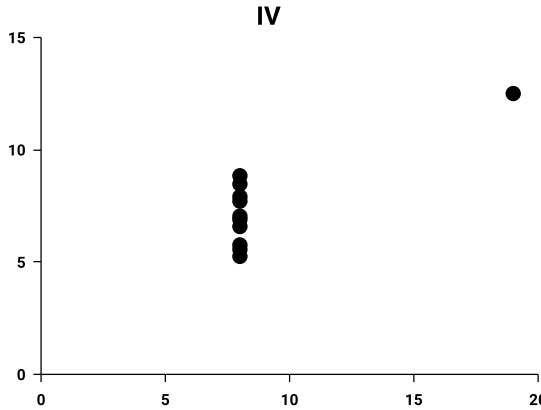
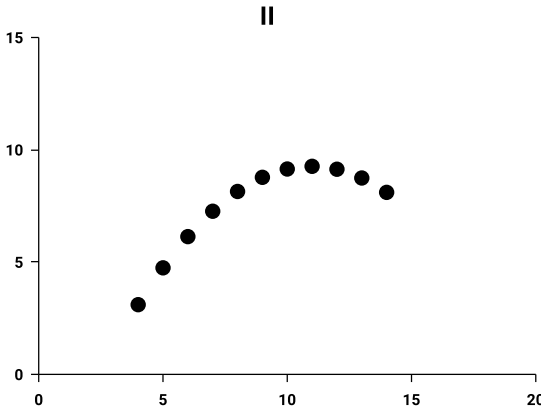
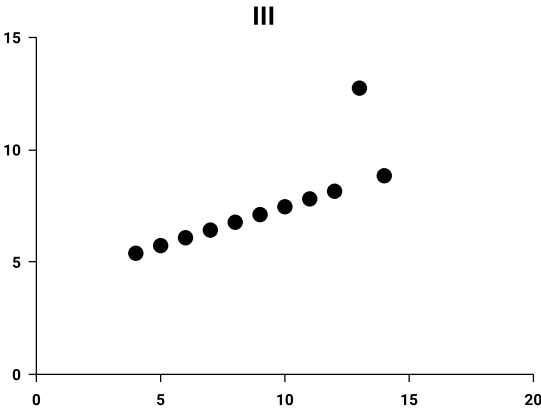
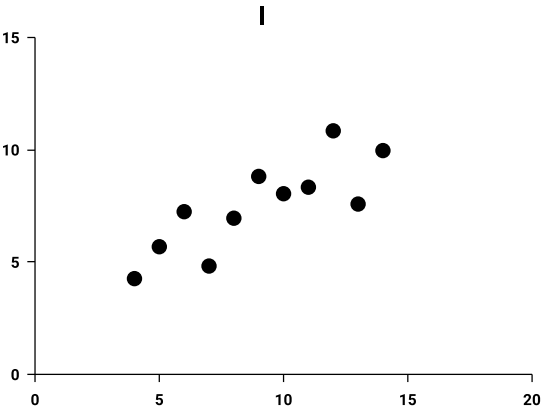
Sumário

1. Introduction
2. The figure structure
3. Graph objects vs Express
4. Creating and updating figures
5. Displaying figures
6. Plot from CSV
- 7. Scatter plots**
8. Line charts
9. Multiple series

Scatter plot

- Sometimes referred to as point graphs or XY scatter graphs, in which observations are plotted in a singular coordinate system.
- Scatter plots are used for exploring relationships, particularly correlations between two or more datasets.

I		II		III		IV		Identical statistics	
x	y	x	y	x	y	x	y		
10.0	8.04	10.0	9.14	10.0	7.46	8.0	6.58	x mean	9.0
8.0	6.95	8.0	8.14	8.0	6.77	8.0	5.76	x variance	10.0
13.0	7.58	13.0	8.74	13.0	12.74	8.0	7.71	y mean	7.50
9.0	8.81	9.0	8.77	9.0	7.11	8.0	8.84	y variance	3.75
11.0	8.33	11.0	9.26	11.0	7.81	8.0	8.47	x/y corelation	0.816
14.0	9.96	14.0	8.10	14.0	8.84	8.0	7.04	N	11
6.0	7.24	6.0	6.13	6.0	6.08	8.0	5.25		
4.0	4.26	4.0	3.10	4.0	5.39	19.0	12.50		
12.0	10.84	12.0	9.13	12.0	8.15	8.0	5.56		
7.0	4.82	7.0	7.26	7.0	6.42	8.0	7.91		
5.0	5.68	5.0	4.74	5.0	5.73	8.0	6.89		
9	7.50	9	7.50	9	7.50	9	7.50	mean	
10.0	3.75	10.0	3.75	10.0	3.75	10.0	3.75	variance	



[example] Anscombe's quartet (Francis Anscombe's 1973 dataset)

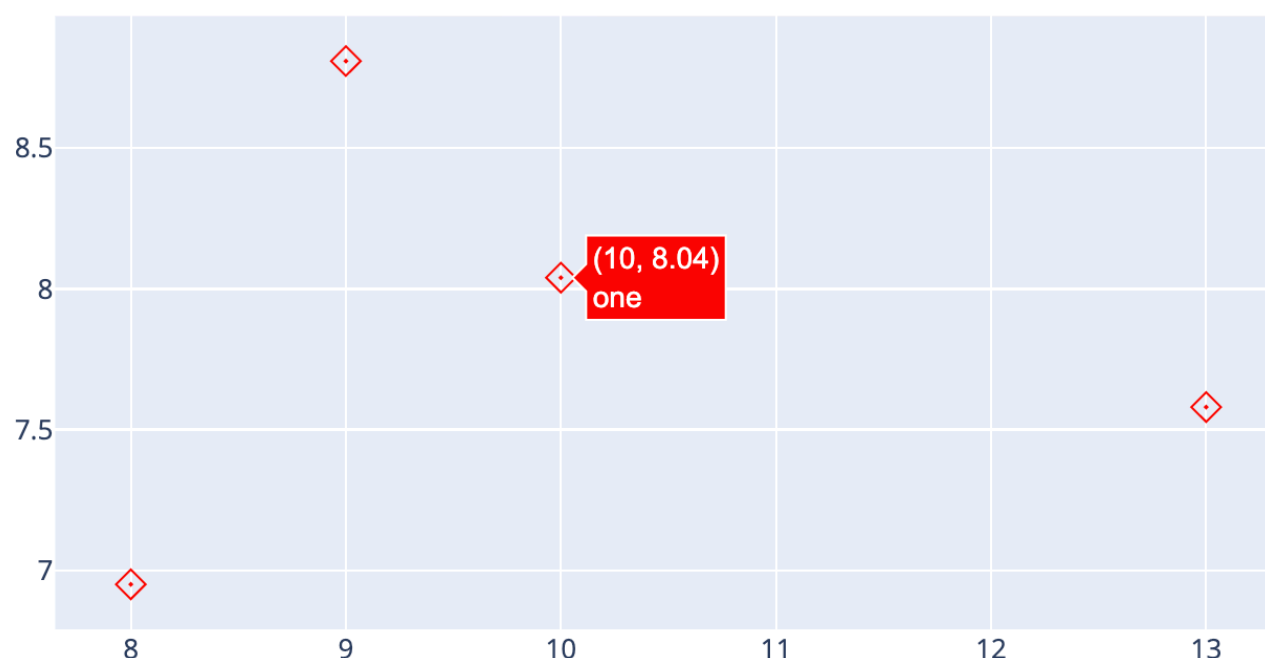
Scatter plot

Plotly

- Consider Anscombe's quartet example.
- In this scenario observation O_1 would be $[x=10.0, y=8.04]$; O_2 would be $[x=8.0, y=6.95]$; and so on.
- Recall, these are called items in dataVis.

```
trace = go.Scatter(x=[10.0, 8.0, 13.0, 9.0], y=[8.04, 6.95, 7.58, 8.81],  
                  marker=dict(color='red', symbol=302, size=10),  
                  mode="markers", text=["one", "two", "three", "four"], name='Anscombe quartet I')
```

I	
x	y
10.0	8.04
8.0	6.95
13.0	7.58
9.0	8.81
11.0	8.33
14.0	9.96
6.0	7.24
4.0	4.26
12.0	10.84
7.0	4.82
5.0	5.68
9	7.50
10.0	3.75



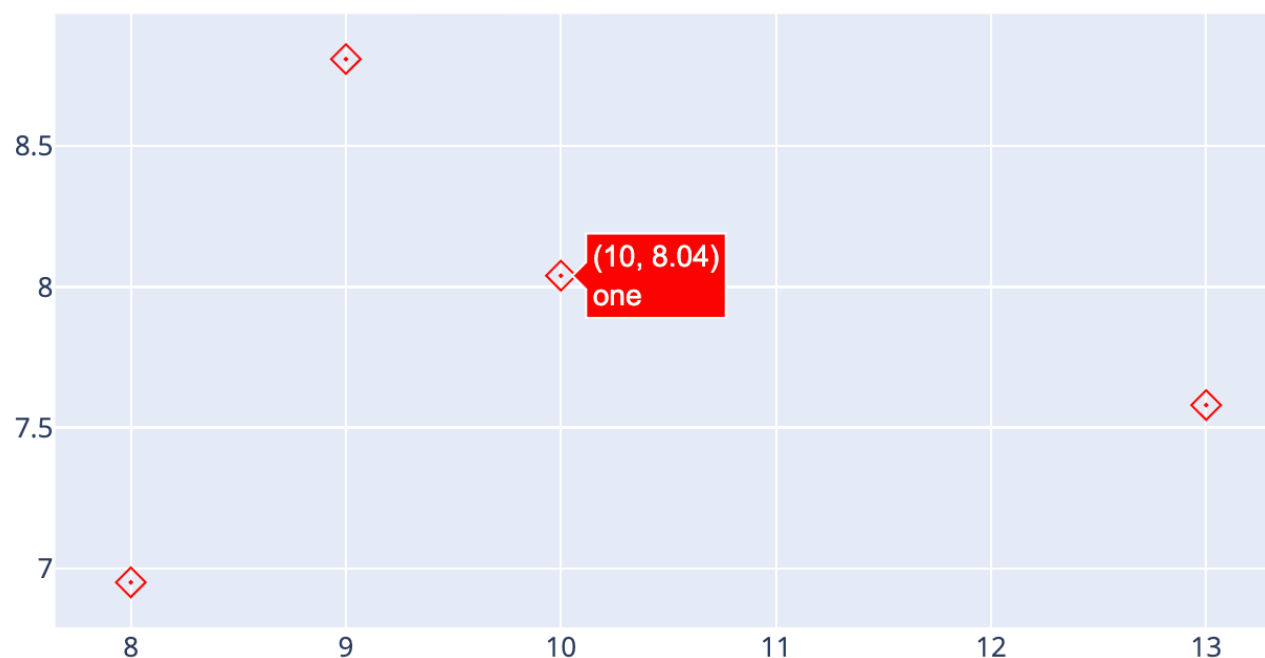
Scatter plot

Plotly

```
trace = go.Scatter(x=[10.0, 8.0, 13.0, 9.0], y=[8.04, 6.95, 7.58, 8.81],  
                  marker=dict(color='red', symbol=302, size=10),  
                  mode="markers", text=["one", "two", "three", "four"], name='Anscombe quartet I')
```

- **X** and **Y** set the corresponding coordinates.
- Can be lists, numpy array or Pandas series of numbers, strings, or datetimes.

I	
x	y
10.0	8.04
8.0	6.95
13.0	7.58
9.0	8.81
11.0	8.33
14.0	9.96
6.0	7.24
4.0	4.26
12.0	10.84
7.0	4.82
5.0	5.68
9	7.50
10.0	3.75

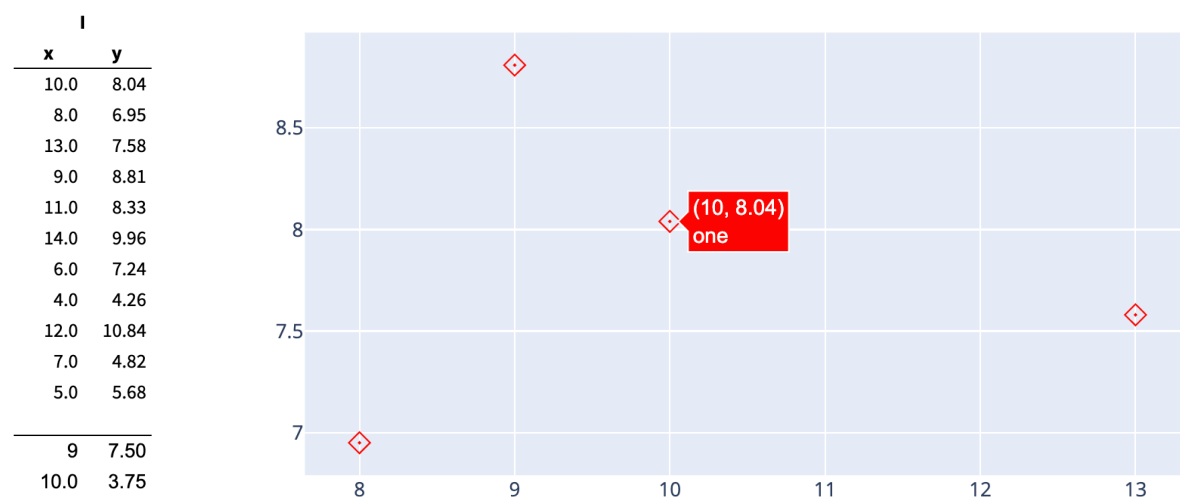


Scatter plot

Plotly

```
trace = go.Scatter(x=[10.0, 8.0, 13.0, 9.0], y=[8.04, 6.95, 7.58, 8.81],  
                  marker=dict(color='red', symbol=302, size=10),  
                  mode="markers", text=["one", "two", "three", "four"], name='Anscombe quartet I')
```

- **Marker** is a dict that defines a visual marker and comprises multiple attributes.
- **Symbol** sets the marker symbol type. Can be a number or a string. Consult the [full table](#)
- **Size** defines the size of markers. Can be a number or array of numbers.
- **Color** defines the color of markers. Can be a color or array of colors. *Colors have special treatment in Plotly and will be covered in detail in future classes.*

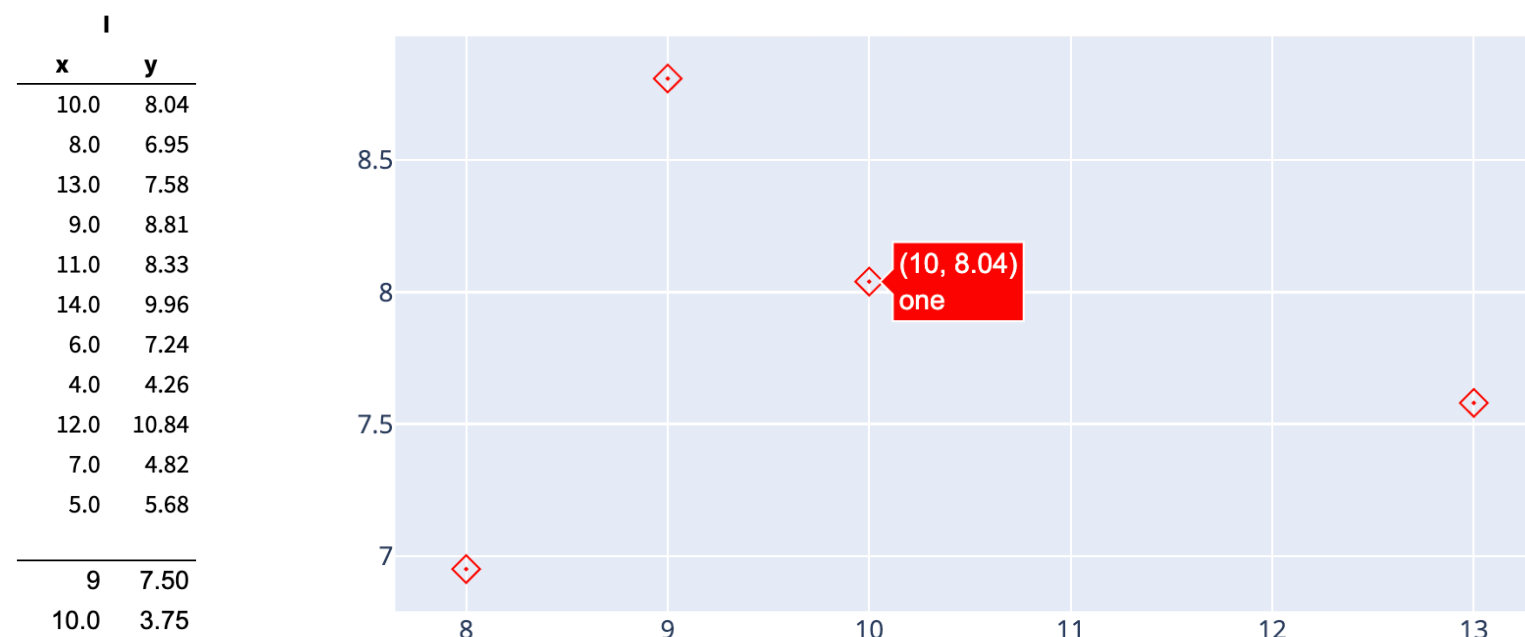


Scatter plot

Plotly

```
trace = go.Scatter(x=[10.0, 8.0, 13.0, 9.0], y=[8.04, 6.95, 7.58, 8.81],  
                  marker=dict(color='red', symbol=302, size=10),  
                  mode="markers", text=["one", "two", "three", "four"], name='Anscombe quartet I')
```

- **Mode** determines the drawing mode. Can be any combination of "lines", "markers", and "text" joined with a "+". Example: "marker+text", "lines+markers"
- **Text** sets the text elements associated with each observation. Can be string or array of strings.
- **Name** sets the trace name, which appear as the legend item and on hover.



Sumário

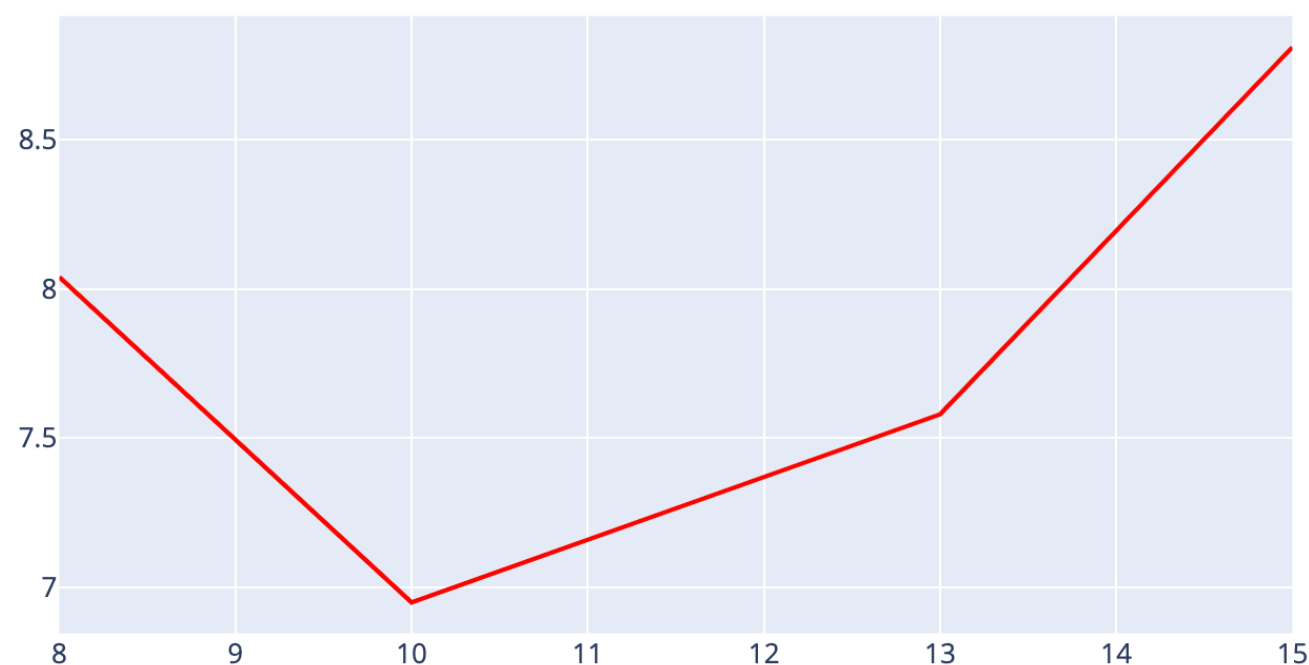
1. Introduction
2. The figure structure
3. Graph objects vs Express
4. Creating and updating figures
5. Displaying figures
6. Plot from CSV
7. Scatter plots
- 8. Line charts**
9. Multiple series

Line charts

Plotly

- Sometimes referred to as line graphs, are a family of graphs that display quantitative information by means of lines.
- In plotly line charts follow the same format as scatter plots with only difference that is the mode "lines".

```
trace = go.Scatter(x=[8.0, 10.0, 13.0, 15.0], y=[8.04, 6.95, 7.58, 8.81],  
                  line=dict(color='red', width=2, dash="solid", shape='linear'),  
                  mode="lines", name='Line chart')
```

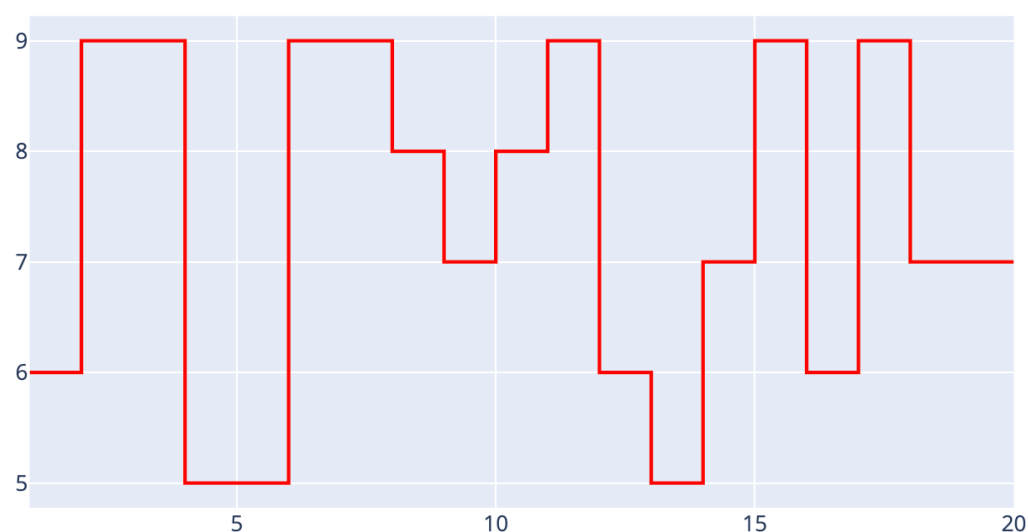


Line charts

Plotly

```
trace = go.Scatter(x=[8.0, 10.0, 13.0, 15.0], y=[8.04, 6.95, 7.58, 8.81],  
                  line=dict(color='red', width=2, dash="solid", shape='linear'),  
                  mode="lines", name='Line chart')
```

- **Line** is a dict that defines visual properties of a line and comprises multiple attributes (details at <https://plotly.com/python/reference/scatter/#scatter-line>)
 - **Width** defines the line thickness (similar to size, in terms of graphical properties)
 - **Dash** sets the dash style of lines. Can be "solid", "dot", "dash", "longdash", "dashdot", or "longdashdot".
 - **Shape** determines the line shape. Can be "linear", "spline", "hv", "vh", "hvh", "vhv". Spline shapes are drawn using spline interpolator.



Step-wise line shapes

Sumário

1. Introduction
2. The figure structure
3. Graph objects vs Express
4. Creating and updating figures
5. Displaying figures
6. Plot from CSV
7. Scatter plots
8. Line charts
- 9. Multiple series**

Multiple series

Plotly

```
month = [ 'January', 'February', 'March', 'April', 'May', 'June', 'July',  
          'August', 'September', 'October', 'November', 'December' ]  
  
high_2007 = [36.5, 26.6, 43.6, 52.3, 71.5, 81.4, 80.5, 82.2, 76.0, 67.3, 46.1, 35.0]  
low_2007 = [23.6, 14.0, 27.0, 36.8, 47.6, 57.7, 58.9, 61.2, 53.3, 48.5, 31.0, 23.6]  
high_2014 = [28.8, 28.5, 37.0, 56.8, 69.7, 79.7, 78.5, 77.8, 74.1, 62.6, 45.3, 39.9]  
low_2014 = [12.7, 14.3, 18.6, 35.5, 49.9, 58.0, 60.0, 58.6, 51.7, 45.2, 32.2, 29.1]  
  
trace1 = go.Scatter(x=month, y=high_2014, name='High 2014',  
                    line=dict(color='firebrick', width=3))  
  
trace2 = go.Scatter(x=month, y=low_2014, name = 'Low 2014',  
                    line=dict(color='royalblue', width=3))  
  
trace3 = go.Scatter(x=month, y=high_2007, name='High 2007',  
                    line=dict(color='firebrick', width=3, dash='dash'),  
                    marker=dict(symbol='x', size=10))  
trace4 = go.Scatter(x=month, y=low_2007, name='Low 2007',  
                    line = dict(color='royalblue', width=3, dash='dash'),  
                    marker=dict(symbol='x', size=10))  
  
layout = go.Layout(title='Average High and Low Temperatures in New York',  
                   xaxis_title='Month',  
                   yaxis_title='Temperature (degrees F)')  
  
fig = go.Figure(data=[trace1, trace2, trace3, trace4], layout=layout)
```

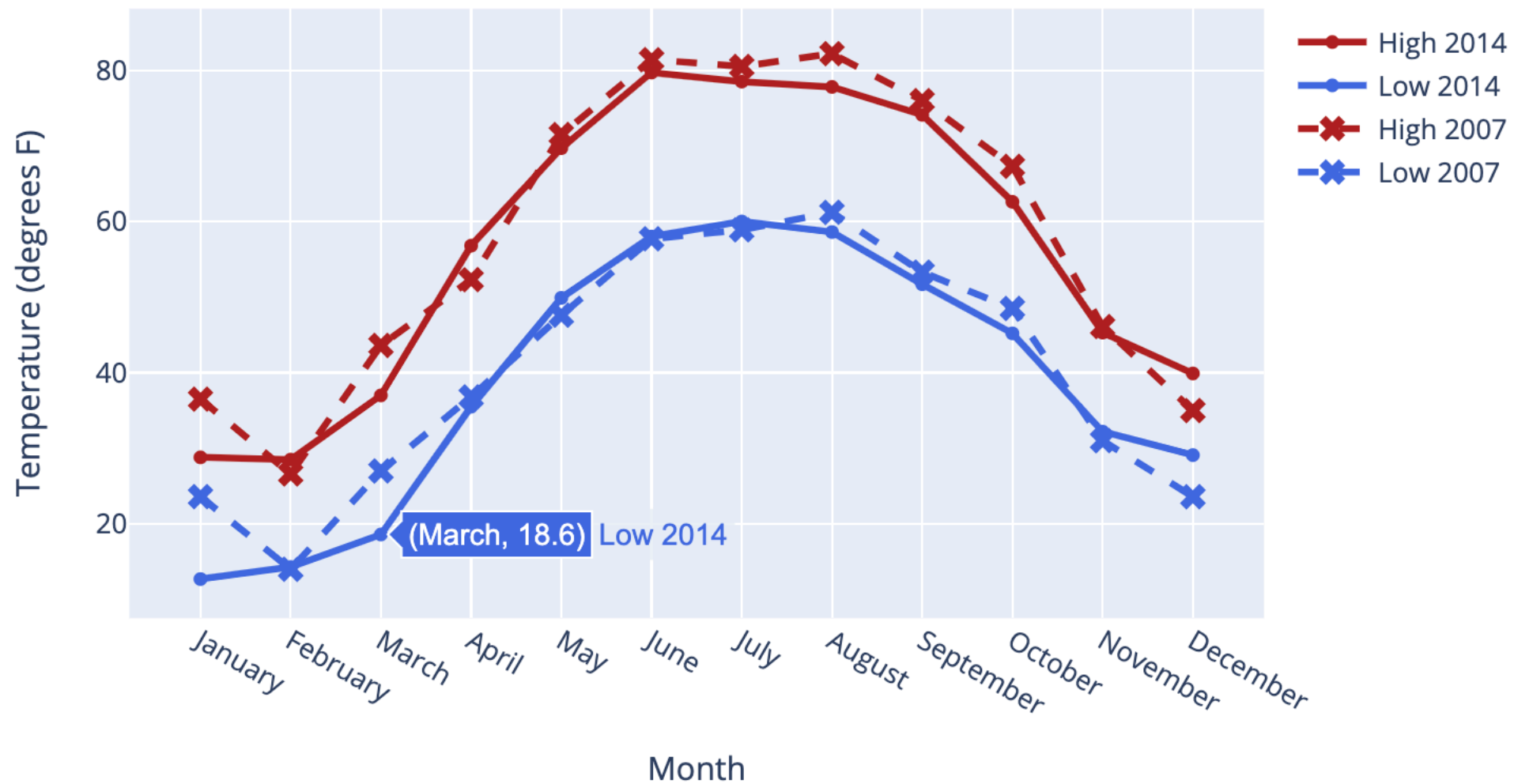
xaxis_title and yaxis_title define respective titles.

Note the use of magic underscores, which indicates that title is an attribute of xaxis class, and alternately can be defined as xaxis=dict(title='Month'). The same applies for yaxis attribute.

Multiple series

Plotly

Average High and Low Temperatures in New York



Multiple series - code simplification

update_*

```
month = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
        'August', 'September', 'October', 'November', 'December']

high_2007 = [36.5, 26.6, 43.6, 52.3, 71.5, 81.4, 80.5, 82.2, 76.0, 67.3, 46.1, 35.0]
low_2007 = [23.6, 14.0, 27.0, 36.8, 47.6, 57.7, 58.9, 61.2, 53.3, 48.5, 31.0, 23.6]
high_2014 = [28.8, 28.5, 37.0, 56.8, 69.7, 79.7, 78.5, 77.8, 74.1, 62.6, 45.3, 39.9]
low_2014 = [12.7, 14.3, 18.6, 35.5, 49.9, 58.0, 60.0, 58.6, 51.7, 45.2, 32.2, 29.1]

trace1 = go.Scatter(x=month, y=high_2014, name='High 2014',
                    line=dict(color='firebrick', width=3))

trace2 = go.Scatter(x=month, y=low_2014, name='Low 2014',
                    line=dict(color='royalblue', width=3))

trace3 = go.Scatter(x=month, y=high_2007, name='High 2007',
                    line=dict(color='firebrick', width=3, dash='dash'),
                    marker=dict(symbol='x', size=10))
trace4 = go.Scatter(x=month, y=low_2007, name='Low 2007',
                    line=dict(color='royalblue', width=3, dash='dash'),
                    marker=dict(symbol='x', size=10))

layout = go.Layout(title='Average High and Low Temperatures in New York',
                   xaxis_title='Month',
                   yaxis_title='Temperature (degrees F)')

fig = go.Figure(data=[trace1, trace2, trace3, trace4], layout=layout)
```

Multiple series - code simplification

`update_*`

```
month = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
         'August', 'September', 'October', 'November', 'December']

high_2007 = [36.5, 26.6, 43.6, 52.3, 71.5, 81.4, 80.5, 82.2, 76.0, 67.3, 46.1, 35.0]
low_2007 = [23.6, 14.0, 27.0, 36.8, 47.6, 57.7, 58.9, 61.2, 53.3, 48.5, 31.0, 23.6]
high_2014 = [28.8, 28.5, 37.0, 56.8, 69.7, 79.7, 78.5, 77.8, 74.1, 62.6, 45.3, 39.9]
low_2014 = [12.7, 14.3, 18.6, 35.5, 49.9, 58.0, 60.0, 58.6, 51.7, 45.2, 32.2, 29.1]

trace1 = go.Scatter(x=month, y=high_2014, name='High 2014',
                    line=dict(color='firebrick'))

trace2 = go.Scatter(x=month, y=low_2014, name='Low 2014',
                    line=dict(color='royalblue'))

trace3 = go.Scatter(x=month, y=high_2007, name='High 2007',
                    line=dict(color='firebrick', dash='dash'),
                    marker=dict(symbol='x'))
trace4 = go.Scatter(x=month, y=low_2007, name='Low 2007',
                    line=dict(color='royalblue', dash='dash'),
                    marker=dict(symbol='x'))

layout = go.Layout(title='Average High and Low Temperatures in New York',
                   xaxis_title='Month',
                   yaxis_title='Temperature (degrees F)')

fig = go.Figure(data=[trace1, trace2, trace3, trace4], layout=layout)

fig.update_traces(line_width=3, marker_size=10, marker_symbol='x'),
```

References

- Figure data structure: <https://plotly.com/python/figure-structure/>
- Creating and updating figures: <https://plotly.com/python/creating-and-updating-figures/>
- Displaying figures: <https://plotly.com/python/renderers/>
- Plotly express: <https://plotly.com/python/plotly-express/>
- Plotly graph objects: <https://plotly.com/python/graph-objects/>
- Python figure reference: <https://plotly.com/python/reference/index/>
- Python API reference: <https://plotly.com/python-api-reference/>
- Plotly datasets: <https://plotly.com/python-api-reference/generated/plotly.data.html#module-plotly.data>